

Functions — a named promise

TC1037 · Algorithmic Thinking · Lecture 04

Dr. Leopoldo Rodríguez Salazar

Friday 2 October 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Sort them, smallest to largest.

And say **exactly** what you are doing, step by step,
so that the person next to you could do it with their eyes closed.

Algorithms to Live By, ch. 3 — *Sorting*

The function you never wrote

```
>>> sorted([9, 4, 1, 7, 2])  
[1, 2, 4, 7, 9]
```

- Somebody wrote that **once** (Tim Peters, 2002 — *Timsort*).
- Gave it a **name**.
- Since then, everyone **calls** it: an input in, an output out, and nobody looks inside.

That is a function. Today you write your own.

Your turn — 3 minutes, on paper

Write your card-sorting procedure as an algorithm.

INPUT: five cards. OUTPUT: the same five, in order. Numbered steps.

Then draw a **box** around the whole thing — and give it a **name**.

You have just written `sort_cards`. Keep the paper; in week 7 you type it.

The smell: the same lines three times

```
area1 = b1 * h1 / 2  
area2 = b2 * h2 / 2  
area3 = b3 * h3 / 2
```

Now the base is in centimetres and the area must be in m^2 . Change the formula.
In all three places. You will miss one.

Not a style problem — a **correctness** problem:
code that exists twice will one day be right in one place and wrong in the other.

def — make the promise

```
1 def triangle_area(base, height):  
2     area = base * height / 2  
3     return area
```

- def · a **name** · **parameters** in parentheses · a colon · an indented body.
- return hands a value back to whoever called — and ends the function.

Nothing has happened yet. A recipe cooks no food until someone follows it.

Calling it — collect on the promise

```
>>> triangle_area(10, 5)
25.0
>>> a = triangle_area(3, 4)
>>> a
6.0
>>> triangle_area(10, 5) + triangle_area(3, 4)
31.0
```

- 10 and 5 are **arguments**; they land in base and height, *in order*.
- The call **is a value** — store it, print it, add it, pass it to another function.

return versus print — the whole idea

✓ returns

```
def area_returns(b, h):  
    return b * h / 2  
  
>>> x = area_returns(10, 5)  
>>> x  
25.0
```

✗ prints

```
def area_prints(b, h):  
    print(b * h / 2)  
  
>>> y = area_prints(10, 5)  
25.0  
>>> y  
None
```

print shows something to a **human**. return gives something to the **program**.

area_prints(10, 5) + 1 → TypeError: ... 'NoneType' and 'int'

Scope — what happens in the function stays in the function

```
1 def triangle_area(base, height):  
2     area = base * height / 2  
3     return area  
4  
5 result = triangle_area(10, 5)  
6 print(area)          # NameError: name 'area' is not defined
```

area was born inside the function and died when it returned.

The only things that cross the boundary: arguments in, the returned value out.

That is a *feature*: you can read `triangle_area` on its own and know everything about it.

main() — the shape of every program from now on

```
1 def triangle_area(base, height):
2     return base * height / 2
3
4 def main():
5     base = float(input("Base: "))
6     height = float(input("Height: "))
7     print("Area:", triangle_area(base, height))
8
9 main()
```

Functions **compute and return** — no input(), no print. main **reads, calls, prints**.

The last line starts it.

The situación problema's final program is specified this way: estadística_descriptiva(),
grafica1(), menu(), main().

A function that decides — and can be tested without input()

```
1 def speed_label(speed):  
2     if speed < 40:  
3         return "slow"  
4     elif speed < 90:  
5         return "normal"  
6     elif speed <= 120:  
7         return "fast"  
8     else:  
9         return "too fast"
```

```
>>> speed_label(39.9)    speed_label(40)    speed_label(120)  
                                speed_label(120.1)
```

One return per branch; each one ends the function.

Second reason functions exist: you test the logic at the boundaries in four seconds, typing nothing in.

Four ways this goes wrong on Tuesday's quick exam

1. **Forgetting to call.** `def` and nothing else — “it doesn't work”. It works; nobody asked it to.
2. `print` **where** `return` **was needed**. Then `total = f(x) + 1` crashes with `NoneType`.
3. **Using a parameter name outside the function.** `NameError`. In through the parentheses, out through `return`.
4. `input()` **inside the function**. It runs — and it cannot be tested or reused. Inputs live in `main`.

Exercise 1 — three functions, one main · 15 min

Lab 1's three programs become three functions in **one** file:

```
triangle_area(base, height)  dollars_to_pesos(dollars, rate)
                             fahrenheit_to_celsius(f)
```

- Each **returns**. None prints. None calls `input()`.
- **Test from the shell first:** run the file, then `triangle_area(3, 4) → 6.0`.
- Then a `main()` that reads the values, calls all three, prints the results. Tests: 25.0 · 1850.0 · 100.0.

Save as `A_Matricula_f1.py`.

Exercise 2 — a function that decides · pairs · 15 min

`classify_triangle(a, b, c)` **returns** one of: "not a triangle" ·
"equilateral" · "isosceles" · "scalene"

Three sides make a triangle only when all are positive and each pair sums to *strictly more* than the third.

(3, 3, 3) equilateral

(3, 3, 5) isosceles

(3, 4, 5) scalene

(1, 2, 3) not a triangle

(0, 4, 4) not a triangle

Algorithm on paper first — *the order of the checks is the algorithm.*

Shell tests before any `main`. Then a `main()` that reads three sides and prints the label.

`A_Matricula_f2.py`

(1, 2, 3): $1 + 2$ is not *greater* than 3. Strict inequality — and the algorithm has to say so.

Before Tuesday

1. **Homework 2 and 3** — both due **Sunday 4 Oct, 23:59**. HW3 is the refactor: your HW2 programs as functions + main, plus `classify_triangle`.
2. **Lab 2 files** — Sunday 23:59 too.
3. Levelling module: all ten lessons by **Thu 8 Oct**. Final diagnostic Fri 9 Oct, in the lab.
4. Read *Self-study: repetition statements* — 10 minutes — so `while` is not new.

Quick exam 2: Tuesday 6 October, first 20 minutes.

Conditionals + functions: read · predict · fix, and one function to write. Rubric on the site.

Functions return. `main` prints.

Tuesday: the program learns to repeat.